

Maths For Games Report

up2193347

May 20, 2025

Contents

1	Artefact Overview	2
1.1	Renderer	2
1.2	Physics Engine	2
2	Maths Library	2
2.1	Vectors	2
2.2	Euler Angles	2
2.3	Matrices	2
2.4	Utilities & Conversions	3
2.5	Debugging	3
3	3D Renderer	3
3.1	Projection	3
3.2	Meshes	3
3.2.1	Winding Order	3
3.3	Lighting	3
4	Physics Engine	3
5	References	4

1 Artefact Overview

For this assignment, I implemented a basic 3D physics engine based on *Game Physics In One Weekend* (Hodges, 2021), and an accompanying OpenGL renderer.

A 30 second video of the artefact running is viewable here:

<https://www.youtube.com/watch?v=blah>.

Add the video

1.1 Renderer

IMAGES!!

I created a basic OpenGL renderer capable of rendering 3D meshes, quads, and text. This required the implementation of projection and transformation matrices, the understanding of the representation of 3D meshes as vertices, normal vectors, and indices, and a basic lighting function.

1.2 Physics Engine

I created a custom physics engine based on *Game Physics In One Weekend* (Hodges, 2021). It features sphere bodies that can collide realistically, and a codebase that can be extended to add further body types. The physics engine is integrated with the renderer to visualise the physics simulation, and provide an interface to add and remove objects from the simulation, and adjust the timescale.

IMAGES!!

2 Maths Library

2.1 Vectors

The core of rendering and physics are **vectors**.

2.2 Euler Angles

Discuss gimbal lock problems, but much more simple to work with.

2.3 Matrices

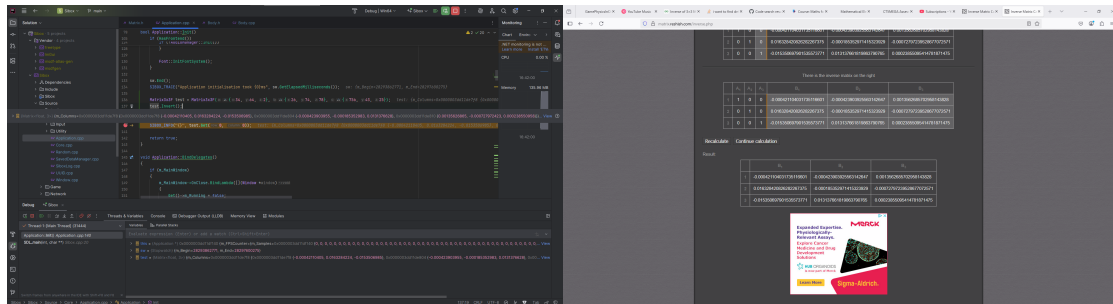
Like vectors, my Matrix type is also templated, but so is the size of the Matrix (although, it is limited to square matrices). I did this to hopefully reduce the amount of lines of code required to implement the types. In hindsight, I believe the templating of the Size mainly increased complexity and hurt performance, as implementations could be less specialised. I avoided this slightly through the use of `if constexpr`, which are if statements that are evaluated at compile-time.

code examples

A consistent issue I encountered with my Matrix class was code that mixed up rows and columns. For example, my initial implementation of matrix multiplication had this issue. I debugged this issue mainly through trial and error. To avoid this issue in the future, I renamed my data variable from `Data` to `Columns`, which made it more explicit in the code that the Matrix data was column-major.

Figure 1

Validating my Matrix 3x3 invert function using an online calculator



2.4 Utilities & Conversions

2.5 Debugging

When debugging, I used an online matrix calculator (Reshish, n.d.) to validate my results. As seen in Figure 1, I was able to validate the correctness of my 3x3 inversion function using a calculator. This was very useful to find where in the library errors were occurring.

In future, it would perhaps be easier to include an established maths library, such as GLM (G-Truc Creation, 2024), in the codebase such that it could be used as a reference for correct outputs; keeping validation within the code would allow the easy use of asserts and the creation of tests.

Debugging was made slightly harder by the use of `FORCE_INLINE`, as breakpoints can behave unpredictably when code is inlined. However, inlining was important for performance. A better implementation may have a toggle to disable `FORCE_INLINE` in debug builds.

3 3D Renderer

3.1 Projection

3.2 Meshes

3.2.1 Winding Order

3.3 Lighting

4 Physics Engine

Explain winding order, relevance to cross products, could solve by flipping mesh indices

Debugging physics. Bodge of multiplying the penetration vector

Discuss performance implications of many conversions between units, types. Rebuild from the top to improve cohesion?

Discuss learning notation

5 References

G-Truc Creation. (2024, February 26). *GLM* (Version 1.0.1). Retrieved May 19, 2025, from <https://github.com/g-truc/glm>

Hodges, G. (2021, January 3). *Game physics in one weekend*.

Reshish. (n.d.). *Inverse matrix calculator*. Retrieved May 19, 2025, from <https://matrix.reshish.com/inverse.php>